Hold (OfficeHours) — Security Writeup

Case Study 3 · CPEG 470

The app in one minute

Flask + Jinja + SQLite office-hours board.

- **Login → cookie.** On a successful login the server inserts a row in `sessions` (a random 48-hex token tied to a user id) and sends it back as the `hold_session` cookie. Every later request, `load_user()` looks that token up and sets `g.user`.
- **Public board vs booking detail.** `/` lists every upcoming slot with no login. A booking detail (`/bookings/<id>`) is private: it shows the student's email and the TA's private note, and the route checks you own that booking.
- **Actions that change server state.** book, cancel, transfer, logout, and (TA) post a slot.
- **Device handoff.** "My bookings" offered a link to open the same session on another browser.

The defect: session token in the URL (session hijacking)

Where: `app.py` — `load_user()` read the session from the query string

```
        token = request.args.get("sid") or
request.cookies.get("hold_session")
```

and `/me` rendered a handoff link containing the live session token:

```
/me?sid=3f97eb97...c53356
```

Why it's bad. The `sid` value *is* the session — the same secret as the cookie, but now sitting in a URL. URLs are not secret: they land in browser history, in server and proxy logs, and in the `Referer` header sent to any off-site link the page loads. The handoff feature even invites the student to copy and share it. Anyone who gets that URL is fully logged in as the student — they can read the private TA note (release codes, makeup-exam info), cancel the hold, or transfer it away. There is no expiry and nothing to revoke.

How to trigger it (observed).

1. Log in as `lea@campus.edu`, open **My bookings**, copy the `…?sid=<token>` link.
2. In a browser with no cookies (or `curl` with no cookie jar), open `/me?sid=<token>`.
3. You land on Lea's bookings and can open `/bookings/1` to read the private note `HOLD-2291`.

```
$ curl "localhost:3000/me?sid=<token>"     # no cookie at all
<title>My bookings · Hold</title>          # logged in as Lea
```

The patch

Keep the handoff feature; stop ever putting the session token in a URL.

1. **Session comes from the cookie only.** Dropped `request.args.get("sid")` from `load_user()`.
2. **One-time handoff code.** New `handoffs` table and two routes:
   - `POST /handoff` mints a random single-use code (shown on its own page, valid 15 minutes).
   - `GET /handoff/<code>` deletes the code, and if it was still valid creates a **new** session for that user. The code is spent on first visit, is not the session token, and expires on its own.
3. **Cookie hardening (same session-theft class).** `hold_session` is now `HttpOnly` (JavaScript can't read it) and `SameSite=Lax`.

```
        # before
        token = request.args.get("sid") or
request.cookies.get("hold_session")
        # after
        token = request.cookies.get("hold_session")
```

A leaked handoff link now exposes at most a 15-minute, one-use code instead of the permanent session key.

Tested, nothing else broken

- Old `?sid=<token>` request with no cookie → **302 to login** (blocked); token no longer appears in the page.
- Login, book a slot, read the private note, transfer to another student, cancel, logout → all still work.
- New device link logs in a fresh browser once, then returns **404** on reuse.

Architecture sketch

```
Browser ──HTTP──> Flask (app.py) ──SQL──> SQLite
(data/officehours.db)
                     routes/sessions          users · slots ·
bookings · sessions · handoffs

Login           POST /login      -> check_password_hash -> INSERT
sessions -> Set-Cookie hold_session (HttpOnly, Lax)
Every request   load_user()      -> read cookie only -> JOIN
sessions/users -> g.user
Public board    GET  /           -> all slots, no auth
Booking detail  GET /bookings/id -> owner-or-TA check -> private
note shown only to the owning student
State changes   POST /slots/id/book · /bookings/id/cancel ·
/bookings/id/transfer · POST /logout
Device handoff  POST /handoff        -> INSERT one-time code (15
min)
                GET  /handoff/<code> -> delete code -> if valid,
INSERT new session -> Set-Cookie
```